Chapter 21

Concurrency Control Techniques

Exam Preparation Notes

Fundamentals of Database Systems — Elmasri & Navathe (7th Ed.)

1. Introduction to Concurrency Control

Concurrency control ensures that simultaneous execution of transactions in a multi-user database results in correct, consistent data — equivalent to some serial execution of those transactions.

1.1 Why Concurrency Control?

- Multiple transactions may execute concurrently in a DBMS
- Without control, they can interfere and corrupt the database
- The goal is to guarantee serializability — the result must equal some serial schedule

1.2 Key Problems Without Concurrency Control

Problem	Description	Example
Lost Update	Two transactions read and overwrite the same item; one update is lost	T1 and T2 both read X=100; T1 writes X=150, T2 writes X=80 — T1's update is lost
Dirty Read	T1 reads data written by T2, but T2 later aborts	T2 writes X=500; T1 reads X=500; T2 rolls back — T1 used invalid data
Unrepeatable Read	T1 reads the same item twice; T2 modifies it between reads	T1 reads X=100 at step 1; T2 writes X=200; T1 reads X=200 at step 3
Phantom Read	T1 executes a query twice; T2 inserts rows between reads	T1 reads all employees with salary>50k; T2 inserts a new one; T1's second read differs

1.3 Overview of Concurrency Control Protocols

Protocol	Core Idea	Deadlock Possible?
Two-Phase Locking (2PL)	Acquire all locks before releasing any	Yes

Timestamp Ordering (TO)	Order operations based on transaction timestamps	No
Multiversion Concurrency Control (MVCC)	Maintain multiple versions of data items	No
Validation / Optimistic CC	Allow free execution; validate before commit	No

2. Two-Phase Locking (2PL)

2.1 What is a Lock?

A lock is a variable associated with a data item that describes the status of that item with respect to possible operations. It prevents conflicting concurrent accesses.

2.2 Binary Locks

- Two states: Locked (1) and Unlocked (0)
- Only one transaction can hold the lock at a time
- Too restrictive — prevents concurrent reads on the same item

Binary Lock Algorithm:

```
lock_item(X):
  if LOCK(X) = 0                  // item is free
    then LOCK(X) = 1              // acquire lock
  else
    wait until LOCK(X) = 0        // block until released
    goto lock_item(X)

unlock_item(X):
  LOCK(X) = 0                     // release lock
  wakeup one waiting transaction  // notify waiters
```

2.3 Shared / Exclusive Locks (Read/Write Locks)

More flexible than binary locks. Allows multiple concurrent readers.

Lock Type	Symbol	Who Can Hold It	Allows Concurrent Access?
Read Lock (Shared)	read_lock(X)	Multiple transactions	Yes — multiple readers allowed
Write Lock (Exclusive)	write_lock(X)	Only one transaction	No — exclusive access only
Unlock	unlock(X)	Holder releases the lock	N/A

Key Rule:
A write_lock can only be granted when LOCK(X) = 'unlocked'. A read_lock can be granted when LOCK(X) = 'unlocked' OR 'read-locked' (since reads don't conflict). A write must wait for ALL readers to finish.

2.4 Lock Conversion

- Upgrading: read_lock → write_lock (during growing phase)
  - Transaction T already holds a read lock and needs to write
  - Must wait until no other transaction holds a read lock on X
- Downgrading: write_lock → read_lock (during shrinking phase)
  - Transaction releases exclusive access but retains shared access

2.5 The Two-Phase Locking Protocol

A transaction follows 2PL if ALL lock operations precede the FIRST unlock operation.

Phase	Name	Allowed Operations
Phase 1	Expanding (Growing) Phase	Acquire new locks; upgrade locks (read→write); CANNOT release locks
Phase 2	Shrinking Phase	Release existing locks; downgrade locks (write→read); CANNOT acquire new locks

Serializability Guarantee:
If EVERY transaction in a schedule follows the 2PL protocol, the resulting schedule is GUARANTEED to be serializable (conflict serializable).

2.6 Example: 2PL in Action

Scenario: X=20, Y=30. T1 computes X := X + Y. T2 computes Y := X + Y.

Schedule following 2PL:

```
T1:  read_lock(Y)    → read Y=30
T1:  read_lock(X)    → read X=20
T1:  write_lock(X)   → (upgrade; waits if needed)
     X := X + Y = 50
T1:  write_item(X)   → X=50
T1:  unlock(Y)       } ← Shrinking phase begins
T1:  unlock(X)       }

T2:  read_lock(X)    → read X=50 (T1 already unlocked)
T2:  read_lock(Y)    → read Y=30
T2:  write_lock(Y)   → upgrade
     Y := X + Y = 80
T2:  write_item(Y)   → Y=80
T2:  unlock(X)
T2:  unlock(Y)

Result: X=50, Y=80  ✓ (same as serial T1 then T2)
```

2.7 Variations of 2PL

Variant	Description	Advantage	Disadvantage
Basic 2PL	Standard growing + shrinking phases	Guarantees serializability	Cascading rollbacks possible
Conservative (Static) 2PL	Lock ALL items before transaction begins (predeclare read-set and write-set)	Deadlock-free	Low concurrency; may block unnecessarily
Strict 2PL	Hold exclusive (write) locks until commit/abort	No dirty reads; no cascading rollbacks	Deadlocks still possible
Rigorous 2PL	Hold ALL locks (read and write) until commit/abort	Simplest to implement; most predictable	Least concurrency; deadlocks possible

3. Deadlock and Starvation

3.1 Deadlock Defined

A deadlock occurs when two or more transactions are each waiting for a lock held by the other, forming a circular wait — none can proceed.

Classic Example:

```
T1: read_lock(Y) — succeeds
T2: read_lock(X) — succeeds
T1: write_lock(X) — WAITS (T2 holds X)
T2: write_lock(Y) — WAITS (T1 holds Y)
→ DEADLOCK: T1 waits for T2, T2 waits for T1
```

3.2 Wait-For Graph

- Nodes = active transactions
- Directed edge T1 → T2 means T1 is waiting for a lock held by T2
- A cycle in the wait-for graph = deadlock detected

3.3 Deadlock Prevention Strategies

Strategy	How It Works	Drawback
Pre-lock all items (Conservative 2PL)	Transaction locks every item it needs before starting	Very low concurrency
Ordered locking	All transactions lock items in a global predefined order	Hard to determine order in advance
Wait-Die (timestamp-based)	If T_old waits for T_new: allowed (old waits for young). If T_new waits for T_old: T_new aborts ('dies')	Younger transactions repeatedly aborted
Wound-Wait (timestamp-based)	If T_old waits for T_new: T_new is aborted ('wounded'). If T_new waits for T_old: allowed (young waits for old)	Older transactions cause aborts of younger ones

Wait-Die vs Wound-Wait Memory Tip:
Wait-Die: Old waits, Young dies. Wound-Wait: Old wounds (kills) young, Young waits for old. Both are deadlock-FREE because circular wait cannot form.

3.4 No-Wait and Cautious-Wait Algorithms

- No-Wait: If a transaction cannot get a lock immediately, it is aborted at once and restarted later. No deadlocks. But high abort rate.
- Cautious-Wait: T1 waits for T2 only if T2 is NOT itself waiting. If T2 is already waiting, T1 aborts. Deadlock-free with fewer aborts than no-wait.

3.5 Deadlock Detection

- System periodically checks for cycles in the wait-for graph
- If a cycle is found, one transaction (the 'victim') is selected and aborted
- Victim selection criteria: least work done, fewest locks held, or lowest priority
- Timeouts: if a transaction waits longer than a threshold, assume deadlock and abort

3.6 Starvation

Starvation occurs when a transaction waits indefinitely because other transactions repeatedly get priority over it.

- Cause: Victim selection always picks the same transaction
- Solution: Use a first-come-first-served (FIFO) queue for lock requests; ensure every transaction eventually gets the lock

4. Timestamp-Based Concurrency Control

4.1 What is a Timestamp?

A timestamp (TS) is a unique identifier assigned by the DBMS to each transaction, representing the time it started. Transactions are ordered by their timestamp.

Property	Detail
Assignment methods	System clock value OR auto-incrementing counter
Uniqueness	No two transactions have the same timestamp
Ordering	TS(Ti) < TS(Tj) means Ti started before Tj
Deadlock	Cannot occur — no locks are used

4.2 Timestamps on Data Items

Each data item X stores two values that the DBMS updates automatically:

- read_TS(X): The largest timestamp among all transactions that have successfully read X
- write_TS(X): The largest timestamp among all transactions that have successfully written X

4.3 Basic Timestamp Ordering (TO) Algorithm

For every operation, the DBMS checks whether it violates the timestamp order:

Rule for read_item(X) by transaction T:

```
IF TS(T) >= write_TS(X):
    Execute read; update read_TS(X) = max(read_TS(X), TS(T))
ELSE:
    Reject read; abort T and restart with a new (later) timestamp
```

Rule for write_item(X) by transaction T:

```
IF TS(T) >= read_TS(X) AND TS(T) >= write_TS(X):
    Execute write; update write_TS(X) = TS(T)
ELSE:
    Reject write; abort T and restart with a new (later) timestamp
```

4.4 Example: Basic TO Algorithm

Transactions: T1 (TS=1), T2 (TS=2). Initial: X=50, read_TS(X)=0, write_TS(X)=0.

```
1. T2: write_item(X=100)
   TS(T2)=2 >= read_TS(X)=0 AND >= write_TS(X)=0 → OK
   write_TS(X) = 2

2. T1: read_item(X)
   TS(T1)=1 < write_TS(X)=2 → REJECT
```

```
   T1 is aborted and restarted with a new timestamp TS=3

3. T1 (now TS=3): read_item(X)
   TS(T1)=3 >= write_TS(X)=2 → OK
   Reads X=100; read_TS(X) = 3
```

4.5 Thomas's Write Rule (Optimization)

A modification of basic TO that allows some 'obsolete' writes to be ignored instead of causing aborts. This improves performance while still producing correct results (view serializable, though not always conflict serializable).

Modified rule for write_item(X) by T:

```
IF TS(T) < read_TS(X):              // too late — someone already read a newer value
    Abort T (same as basic TO)
ELSE IF TS(T) < write_TS(X):        // another transaction already wrote a newer value
    Ignore this write (skip it)      // ← KEY DIFFERENCE from basic TO
    Do NOT abort T; just skip the write
ELSE:
    Execute write; write_TS(X) = TS(T)
```

Thomas's Write Rule Benefit:
In basic TO, if TS(T) < write_TS(X), T is aborted. In Thomas's rule, the write is simply ignored (not aborted), because a newer version already exists and T's write is 'obsolete'. This reduces unnecessary aborts.

4.6 Strict Timestamp Ordering

- Extension of basic TO
- A transaction T that reads/writes X must wait until all transactions with earlier timestamps that have written X have committed or aborted
- Ensures schedules are both strict AND conflict serializable
- Avoids cascading aborts (dirty reads from uncommitted data)

4.7 Comparison: 2PL vs Timestamp Ordering

Feature	Two-Phase Locking	Timestamp Ordering
Mechanism	Locks on data items	Timestamps on transactions
Deadlock	Possible (needs prevention/detection)	Impossible
Starvation	Possible	Possible (repeated aborts)
Serializability	Conflict serializable	Conflict serializable (basic TO)
Cascading Abort	Possible in basic 2PL; avoided in Strict 2PL	Possible in basic TO; avoided in Strict TO

Overhead	Lock management overhead	Timestamp comparison overhead

5. Multiversion Concurrency Control (MVCC)

5.1 Core Idea

Instead of overwriting data, the DBMS keeps multiple versions of each data item. Each write creates a new version. Reads can be served from an older version without blocking writers.

- Each version has a read_TS and write_TS attached
- Greatly reduces read-write conflicts
- Used in many real-world DBMSs: PostgreSQL, Oracle, MySQL InnoDB

5.2 Advantages and Disadvantages

Advantage	Disadvantage
Readers never block writers	More storage needed for multiple versions
Writers never block readers	Garbage collection needed to remove old versions
Higher concurrency	More complex to implement

6. Optimistic (Validation-Based) Concurrency Control

6.1 Philosophy

Assumes conflicts are RARE. Transactions execute freely without locks or checks. Before committing, a validation phase checks for conflicts. If a conflict is found, the transaction is aborted and restarted.

6.2 Three Phases of Optimistic CC

Phase	Activity
1. Read Phase	Transaction executes, reads from database, writes to a local copy (workspace). No locks held.
2. Validation Phase	System checks whether the transaction's local writes conflict with any other committed transaction.
3. Write Phase	If validation passes, changes are written to the actual database. If validation fails, the transaction is aborted.

When is Optimistic CC Best?
When conflicts are rare (read-heavy workloads, short transactions). It avoids all lock overhead. It performs poorly under high contention because many transactions may fail validation and be restarted.

7. Quick-Reference Summary Table

Topic	Key Formula / Rule / Fact
Binary Lock	0 = unlocked, 1 = locked. Only one holder at a time.
Shared Lock	Multiple readers allowed simultaneously
Exclusive Lock	Only one writer; must wait for all readers to release
2PL Rule	All lock operations before first unlock
Expanding Phase	Only acquire/upgrade locks — no releases
Shrinking Phase	Only release/downgrade locks — no new acquisitions
Conservative 2PL	Lock all items before start → deadlock-free
Strict 2PL	Hold write locks until commit/abort → no dirty reads
Rigorous 2PL	Hold ALL locks until commit/abort
Deadlock	Cycle in wait-for graph
Wait-Die	Old waits; Young dies. Deadlock-free.
Wound-Wait	Old wounds young (aborts it); Young waits. Deadlock-free.
TS(T) < write_TS(X) on read	Abort T (basic TO)
TS(T) < write_TS(X) on write	Abort T (basic TO); Ignore write (Thomas's rule)
TS(T) < read_TS(X) on write	Abort T (both basic TO and Thomas's rule)
Starvation fix	FIFO queue for lock requests
MVCC	Multiple data versions; reads never block writes

8. Exam-Style Questions with Solutions

Section A: Short Answer / MCQ-Level

Q1. What are the two phases of the Two-Phase Locking protocol? What is allowed/forbidden in each?

Answer:

Phase 1 — Expanding (Growing): A transaction may acquire new locks and upgrade existing locks (read → write). It CANNOT release any lock. Phase 2 — Shrinking: A transaction may release locks and downgrade them (write → read). It CANNOT acquire any new locks. The key rule: the first unlock operation marks the end of the growing phase and the start of the shrinking phase.

Q2. What is the difference between Strict 2PL and Rigorous 2PL?

Answer:

Strict 2PL: A transaction holds its exclusive (write) locks until it commits or aborts. Read locks can be released earlier. This prevents dirty reads and cascading aborts. Rigorous 2PL: A transaction holds ALL locks (both read and write) until it commits or aborts. This is the most restrictive variant but the simplest to implement correctly. It also prevents dirty reads and cascading aborts.

Q3. Why does the Timestamp Ordering protocol never experience deadlocks?

Answer:

In timestamp ordering, no locks are used. When a transaction's operation would violate timestamp order, the transaction is simply ABORTED and RESTARTED with a new, later timestamp. Since no transaction ever 'waits' for another transaction to release a lock, the circular wait condition required for deadlock can never form.

Q4. Explain Thomas's Write Rule and how it differs from the Basic Timestamp Ordering algorithm.

Answer:

In Basic TO, if a transaction T issues write_item(X) and TS(T) < write_TS(X), T is aborted because a newer transaction has already written X. Thomas's Write Rule modifies this: instead of aborting T, the write is simply IGNORED (skipped), because the value T wants to write is already 'obsolete' — a newer version exists. T is not aborted and can continue. However, if TS(T) < read_TS(X), T must still be aborted (someone already read the current version, so T's write would be inconsistent). Thomas's rule produces view-serializable (but not necessarily conflict-serializable) schedules, and results in fewer transaction aborts.

Section B: Scenario / Trace Questions

Q5. Given transactions T1 (TS=5) and T2 (TS=10), and data item X with read_TS(X)=8 and write_TS(X)=3: (a) Can T1 read X? (b) Can T2 write X? (c) Can T1 write X using basic TO? (d) Can T1 write X using Thomas's Write Rule?

Answer:

(a) T1 reads X: TS(T1)=5 vs write_TS(X)=3. Since 5 >= 3, T1 CAN read X. read_TS(X) becomes max(8,5) = 8. (b) T2 writes X: TS(T2)=10 vs read_TS(X)=8 (10>=8 ✓) AND vs write_TS(X)=3 (10>=3 ✓). T2 CAN write X. write_TS(X) becomes 10. (c) T1 writes X (Basic TO): TS(T1)=5 vs read_TS(X)=8. Since 5 < 8, T1's write is REJECTED and T1 is aborted. (d) T1 writes X (Thomas's

Rule): TS(T1)=5 vs read_TS(X)=8. Since 5 < 8, T1 is still aborted. (Thomas's rule only helps when TS(T) < write_TS(X) but TS(T) >= read_TS(X) — in that case the write is ignored rather than aborting T.)

Q6. Identify whether the following transaction follows the 2PL protocol: read_lock(A); read_item(A); write_lock(B); write_item(B); unlock(A); read_lock(C); read_item(C); unlock(B); unlock(C)

Answer:

NO, this transaction does NOT follow 2PL. After unlock(A) (step 5), the transaction acquires read_lock(C) (step 6). This is acquiring a new lock AFTER releasing a lock — which violates the 2PL rule that the shrinking phase cannot include new lock acquisitions. Correct 2PL would require all lock acquisitions to occur before any unlock. For example: read_lock(A); write_lock(B); read_lock(C); read_item(A); write_item(B); read_item(C); unlock(A); unlock(B); unlock(C) ← Valid 2PL

Q7. Describe the Wait-Die and Wound-Wait deadlock prevention schemes. If T1 has TS=5 and T2 has TS=10, and T1 needs a lock held by T2: (a) What happens under Wait-Die? (b) What happens under Wound-Wait?

Answer:

Wait-Die Rule: If Ti needs a lock held by Tj: - If TS(Ti) < TS(Tj) — Ti is OLDER — Ti WAITS - If TS(Ti) > TS(Tj) — Ti is YOUNGER — Ti DIES (aborted, restarted) Wound-Wait Rule: If Ti needs a lock held by Tj: - If TS(Ti) < TS(Tj) — Ti is OLDER — Ti WOUNDS Tj (Tj is aborted) and Ti gets the lock - If TS(Ti) > TS(Tj) — Ti is YOUNGER — Ti WAITS (a) T1 (TS=5) needs lock held by T2 (TS=10). T1 is older (5 < 10). Under Wait-Die: T1 WAITS. (b) T1 (TS=5) needs lock held by T2 (TS=10). T1 is older. Under Wound-Wait: T1 WOUNDS T2 — T2 is aborted, T1 gets the lock.

Q8. A schedule has the following operations on item X (initial read_TS(X)=0, write_TS(X)=0): T3(TS=15): write_item(X) T1(TS=5): read_item(X) T2(TS=10): write_item(X) Apply Basic TO and determine which transactions are aborted.

Answer:

Step 1 — T3 (TS=15) writes X: TS(T3)=15 >= read_TS(X)=0 ✓ AND >= write_TS(X)=0 ✓ Execute write. write_TS(X) = 15. Step 2 — T1 (TS=5) reads X: TS(T1)=5 < write_TS(X)=15 ✗ ABORT T1. T1 is restarted with a new larger timestamp. Step 3 — T2 (TS=10) writes X: TS(T2)=10 < write_TS(X)=15 ✗ (and also check read_TS(X)=0: 10>=0 ✓ but write_TS check fails) Under Basic TO: ABORT T2. Under Thomas's Write Rule: TS(T2)=10 >= read_TS(X)=0 ✓, but TS(T2)=10 < write_TS(X)=15 → write is IGNORED (T2 not aborted, continues). Conclusion (Basic TO): T1 and T2 are aborted. T3 succeeds. Conclusion (Thomas's Rule): Only T1 is aborted. T2 continues (its write skipped). T3 succeeds.

Section C: Compare / Discuss

Q9. Compare Conservative 2PL and Strict 2PL. When would you prefer each?

Answer:

Conservative (Static) 2PL: - Transaction must lock ALL items it will ever need BEFORE it begins execution - Advantage: Completely DEADLOCK-FREE - Disadvantage: Requires knowing all data items in advance (often impractical); holds locks for a long time reducing concurrency - Use when: transaction data access patterns are known in advance and deadlock avoidance is critical Strict 2PL: - Locks can be acquired as needed during execution - Exclusive (write) locks held until

transaction commits or aborts - Advantage: Prevents dirty reads and cascading rollbacks; deadlocks still prevented in practice with detection - Disadvantage: Deadlocks are possible; slightly more complex - Use when: need to prevent dirty reads and cascading aborts in a concurrent environment (most common in practice) Strict 2PL is more widely used in real DBMS implementations because it is more practical while still preventing the most harmful anomalies.

Q10. What is starvation in the context of concurrency control, and how is it prevented?

Answer:

Starvation occurs when a transaction is repeatedly denied access to a data item — or is repeatedly selected as a victim during deadlock resolution — and therefore never completes, even though other transactions continue normally. Causes: - Deadlock victim selection always picks the same transaction (e.g., youngest or least-work-done) - Write requests starved by a continuous stream of read requests (readers block writers indefinitely) Prevention: - Use a FIFO (first-come-first-served) queue for lock requests: the transaction that has been waiting longest gets priority - Track how many times a transaction has been aborted; give higher priority to transactions that have been waiting longer - Use fairness policies in the lock manager to ensure every transaction eventually proceeds

— End of Chapter 21 Notes —